

GSHTrans: mathematics and numerical methods

A reference for what the library does, and why

Abstract

This is the mathematical and numerical reference for GSHTrans. Part I collects the definitions, conventions and relations the library rests on — the canonical basis and canonical components, the generalized spherical harmonics, reality and which components are stored, the operations that are admissible on components, raising and lowering, and the contravariant derivative — in the form the library actually uses them, with a translation table for the three sources whose notations differ. Part II explains the numerical methods: what the algorithms are, why they were chosen, how they are stored and threaded, and the limits that have been measured. Part III states what the type system of the field algebra enforces and why, and what each test oracle pins. The document closes with the scope of the library and the reasons for what it deliberately does not do.

How to use the library is in `README.md` and the examples; the software design — the layering, the interfaces and the reasons for them — is in `docs/design.md`; work that is planned or open is tracked in `docs/open-issues.md`. Statements here are either verified by a test in the library or read directly from a cited source.

Contents

I	Mathematics	3
1	Conventions	3
1.1	The canonical basis	3
1.2	What the library stores, exactly	3
1.3	Translation table	4
1.4	What remains a convention	4
2	Canonical components	5
2.1	Admissible operations	5
3	Generalized spherical harmonics	6
4	The generalized Legendre functions	7
4.1	Symmetries	7
4.2	Boundary values	7
4.3	The addition theorem	7
5	Reality	7
5.1	Which components must be stored	8
5.2	Interaction with permutation symmetry	8
5.3	Reality in the spectral domain	9
6	Raising and lowering	9

7	The contravariant derivative, and how it differs	10
7.1	The definition	10
7.2	Two differences, one benign and one not	10
7.3	The consequence for this library	11
7.4	The intrinsic derivative, where $\mathfrak{D}$ is exactly right	11
7.5	What is implemented	11
8	Quadrature	12
II	Numerical methods	13
9	The transform	13
9.1	The algorithm	13
9.2	Real fields	13
9.3	Transforming is a projection	14
10	Computing the d-functions	14
10.1	Recursion in l , not in m	14
10.2	The seed row and the boundary terms	15
10.3	Why the recursion is the safer form	15
10.4	The poles, and why no $\log \Gamma$ is evaluated on a threaded path	16
11	Storage and access	16
11.1	The table	16
11.2	The row supplier	17
12	Batching, chunking and threading	17
12.1	The batch	17
12.2	Chunking	18
12.3	Threading	18
13	The matrix kernel	19
13.1	The Legendre stage is a matrix product	19
13.2	Why it is faster	19
13.3	Storage layout and the Fourier stage	20
13.4	Threading	20
13.5	Measured performance	21
13.6	What it composes with	21
14	Interpolation	21
15	The 3-j symbols	22
16	Choosing a policy by measurement	23
III	The field algebra	23
17	What the type system enforces	24
17.1	The index algebra	24
17.2	Real-valuedness	24
17.3	Laziness, and the aliasing theorem	25
17.4	Which components a tensor stores	25

17.5 Derived components in the two domains	25
18 Validation	26
19 Scope and limitations	26

Part I

Mathematics

1 Conventions

1.1 The canonical basis

At each point of the sphere, with $\hat{\mathbf{r}}, \hat{\boldsymbol{\theta}}, \hat{\boldsymbol{\phi}}$ the usual orthonormal spherical polar frame, the canonical basis is

$$\mathbf{e}_{\pm} = \mp \frac{1}{\sqrt{2}} (\hat{\boldsymbol{\theta}} \pm i \hat{\boldsymbol{\phi}}), \quad \mathbf{e}_0 = \hat{\mathbf{r}}, \quad \alpha \in \{-1, 0, +1\}, \quad (1)$$

the index α being written $\{-, 0, +\}$ where no confusion arises. Two consequences are used everywhere. Complex conjugation of the basis is

$$\overline{\mathbf{e}_{\alpha}} = (-1)^{\alpha} \mathbf{e}_{-\alpha}, \quad (2)$$

which reproduces $\overline{\mathbf{e}_{\pm}} = -\mathbf{e}_{\mp}$ and $\overline{\mathbf{e}_0} = \mathbf{e}_0$; and the metric is

$$g_{\alpha\beta} = \mathbf{e}_{\alpha} \cdot \mathbf{e}_{\beta} = (-1)^{\alpha} \delta_{\alpha+\beta, 0}, \quad \text{that is} \quad g_{+-} = g_{-+} = -1, \quad g_{00} = 1, \quad (3)$$

all other entries vanishing. g is its own inverse. The library works with contravariant components throughout and applies g explicitly wherever a contraction is taken; index position is not tracked.

Under a rotation of the tangent frame through ψ about $\hat{\mathbf{r}}$, $\mathbf{e}_{\pm} \mapsto e^{\mp i\psi} \mathbf{e}_{\pm}$ and $\mathbf{e}_0 \mapsto \mathbf{e}_0$.

1.2 What the library stores, exactly

The single most useful statement for reading the code:

Wigner's stored value at upper index N , degree l and order m is

$$\left(\frac{2l+1}{4\pi} \right)^{1/2} d_{Nm}^l(\theta) = X_{lm}^N(\theta), \quad (4)$$

which is exactly the scalar harmonic X_{lm}^N of Dahlen & Tromp (1998), their (C.117), built on their generalized Legendre function $P_{lm}^N = d_{Nm}^l$ — *upper index first*.

This is verified rather than assumed. `tests/CheckWignerConvention.hpp` compares all nine $l = 1$ values against D&T (C.115); the check discriminates, because the four entries with $N - m$ odd distinguish d_{Nm}^l from d_{mN}^l , which differ by $(-1)^{N-m}$. Two further facts corroborate it: the recursion seed evaluates to D&T (C.125), and `tests/CheckLegendre.hpp` pins the $N = 0$ row against `std::sph_legendre`, fixing the orthonormal scaling and the Condon–Shortley phase.

1.3 Translation table

The three standard sources use different notations, and two of them differ from this library in normalisation. Everything in this table is read directly from the cited equations.

	harmonic	colatitudinal part	normalised?
GSHTrans	$Y_{lm}^N = X_{lm}^N e^{im\phi}$	$X_{lm}^N = \left(\frac{2l+1}{4\pi}\right)^{1/2} d_{Nm}^l$	yes
Dahlen & Tromp (C.117)	$Y_{lm}^N = X_{lm}^N e^{im\phi}$	$X_{lm}^N = \left(\frac{2l+1}{4\pi}\right)^{1/2} P_{lm}^N$	yes
Woodhouse & Deuss [41]	$Y_l^{Nm} = P_l^{Nm} e^{im\phi}$	$P_l^{Nm} = d_{Nm}^{(l)}$	no
Phinney & Burridge (1.5)	$X^{Nm} = P_l^{Nm} e^{im\phi}$	P_l^{Nm} , their Appendix A	no

Three things to take from it.

The index order is the same in all four. D&T write P_{lm}^N , Woodhouse and P&B write P_l^{Nm} , and this library writes d_{Nm}^l ; all mean the same function with the upper index first. D&T state that they follow P&B in the definition.

The normalisation is not. D&T and this library carry the $[(2l+1)/4\pi]^{1/2}$; Woodhouse and P&B do not. A formula taken from Woodhouse or P&B therefore needs that factor supplied, and a formula taken from D&T does not.

The transposed function is a different thing. D&T (C.118) gives $P_{lN}^m = (-1)^{m+N} P_{lm}^N$, so getting the index order wrong costs a sign whenever $m - N$ is odd, and nothing when it is even — which is exactly the kind of error that survives half a test suite.

1.4 What remains a convention

Two signs are matters of convention in the literature: the $\mp$ in (1), since some authors use $\mathbf{e}_{\pm} = \pm(\hat{\boldsymbol{\theta}} \pm i\hat{\boldsymbol{\phi}})/\sqrt{2}$, which flips every odd-rank reality phase; and the overall signs of $\tilde{\partial}$ and $\bar{\partial}$ (§6). Neither is observable from anything that stays in canonical components from end to end. Both become observable, jointly, once a result is read back into the physical frame, and the library pins them by requiring a gradient to be the gradient. Inverting (1) for a vector $\mathbf{v} = v^{\alpha} \mathbf{e}_{\alpha}$,

$$v_{\theta} = \frac{v^{-} - v^{+}}{\sqrt{2}}, \quad v_{\phi} = -i \frac{v^{+} + v^{-}}{\sqrt{2}}, \quad v_r = v^0, \quad (5)$$

and `tests/TestConventions.cpp` requires that for a scalar f the surface gradient satisfies $\mathbf{v} = \hat{\boldsymbol{\theta}} \partial_{\theta} f + \hat{\boldsymbol{\phi}} (\sin \theta)^{-1} \partial_{\phi} f$ pointwise, and that the metric trace of the surface gradient of a tangential vector field is its surface divergence. Both hold to 10^{-13} , and both fail by an $O(1)$ amount if either sign is flipped alone.

What is left is genuinely a convention and nothing more: flipping (1) *and* the sign of the $\tilde{\partial}$ pair together is a relabelling of which component is called +1, and (5) flips with it. So there is one convention here rather than two unknowns, this document states it, and a test observes it.

A trap in constructing test fields. A tangential field written as $a(\theta, \phi) \hat{\boldsymbol{\theta}}$ is almost never smooth, because $\hat{\boldsymbol{\theta}}$ is not: at a pole its direction depends on the azimuth from which the pole is approached. The innocuous-looking $a = \sin \theta \cos \phi$ is not differentiable there, its spin-weighted expansion does not converge, and the divergence check above applied to it is wrong by 0.18 at every truncation from $l_{\max} = 8$ to 128. That is what a non-convergent expansion looks like, and it is easily mistaken for a bug in the operator. The test fields are built as $\nabla_1 f + \hat{\mathbf{r}} \times \nabla_1 g$ from smooth scalars, which is smooth by construction.

2 Canonical components

A rank- p tensor field on the sphere is expanded as

$$\mathbf{T}(\theta, \phi) = \sum_{\alpha_1 \dots \alpha_p} T^{\alpha_1 \dots \alpha_p}(\theta, \phi) \mathbf{e}_{\alpha_1} \otimes \dots \otimes \mathbf{e}_{\alpha_p}, \quad (6)$$

so it has 3^p scalar components, each labelled by a *multi-index* $(\alpha_1 \dots \alpha_p) \in \{-1, 0, 1\}^p$. Under the frame rotation of §1.1, the component $T^{\alpha_1 \dots \alpha_p}$ acquires the phase $e^{-iN\psi}$ with

$$N = \sum_{i=1}^p \alpha_i, \quad (7)$$

and it is this N — not the multi-index — that is the upper index of the component's harmonic expansion (§3).

The multi-index is not the upper index. For a vector they coincide, $N = \alpha$, which is the coincidence that misleads. For $p \geq 2$ distinct components share an upper index: a rank-2 tensor has three at $N = 0$, namely $(-+)$, (00) and $(+-)$. A collection of components labelled only by N therefore does *not* determine a tensor. The number of components of rank p carrying a given N is the trinomial coefficient $\binom{p}{N}_2$; for $p = 2$ these are 1, 2, 3, 2, 1 across $N = -2 \dots 2$:

N	rank-2 components	count
-2	$(--)$	1
-1	$(-0), (0-)$	2
0	$(-+), (00), (+-)$	3
+1	$(0+), (+0)$	2
+2	$(++)$	1

Tensor products. If S has multi-index $(\alpha_1 \dots \alpha_p)$ with upper index N_S and T has $(\beta_1 \dots \beta_q)$ with N_T , then the component of $S \otimes T$ with the concatenated multi-index has upper index $N_S + N_T$ by (7). The rule “upper indices add under pointwise multiplication” is exactly this statement.

Contraction. Contracting slots j and k against the metric (3) gives

$$(\text{tr}_{jk} T)^{\dots} = \sum_{\alpha\beta} g_{\alpha\beta} T^{\dots\alpha\dots\beta\dots} = \sum_{\alpha} (-1)^{\alpha} T^{\dots\alpha\dots-\alpha\dots} = -T^{\dots- \dots+ \dots} + T^{\dots 0 \dots 0 \dots} - T^{\dots+ \dots- \dots}, \quad (8)$$

the contracted pair contributing $\alpha + (-\alpha) = 0$, so the result carries the upper index of the surviving slots, as it must.

2.1 Admissible operations

An operation on canonical components is admissible only if its result is again a component of definite upper index, that is, only if it commutes with the frame rotation $\mathbf{e}_{\pm} \mapsto e^{\mp i\psi} \mathbf{e}_{\pm}$. Writing $[f] = N$ for the upper index of f :

operation	upper index of result	restriction
$f + g, f - g$	N	requires $[f] = [g] = N$
fg	$[f] + [g]$	—
f/g	$[f]$	requires $[g] = 0$
$sf, s \in \mathbb{C}$	$[f]$	—
$\bar{f}$	$-[f]$	—
$ f , f ^2$	0	—
$\text{Re } f, \text{Im } f$	0	requires $[f] = 0$
$F(f), F$ nonlinear	0	requires $[f] = 0$
$\int_{S^2} f \, d\Omega$	—	requires $[f] = 0$

Three of these are easy to get wrong, and are worth stating.

- **Conjugation reverses the upper index.** A quantity carrying $e^{-iN\psi}$ has a conjugate carrying $e^{+iN\psi}$. This is consistent with the reality condition (17): u^- , of upper index -1 , is $-u^+$, and u^+ has upper index $+1$.
- **Real and imaginary parts are defined only at $N = 0$.** For $N \neq 0$ the split of f into real and imaginary parts is not preserved by a frame rotation, so neither part is a component of any tensor. What *is* available at any N is the modulus, which is a genuine scalar.
- **Nonlinear functions likewise require $N = 0$.** Only for $N = 0$, where the component is invariant, is $F(f)$ again a component. Integration over the sphere is subject to the same restriction, since $\int f \, d\Omega$ vanishes identically unless $N = 0$.

It follows that a *real-valued* component field can occur only at $N = 0$: real-valuedness is not preserved by the frame rotation anywhere else.

3 Generalized spherical harmonics

A field f of upper index N expands as

$$f(\theta, \phi) = \sum_{l \geq |N|} \sum_{m=-l}^l f_{lm}^N Y_{lm}^N, \quad f_{lm}^N = \int_{S^2} \bar{Y}_{lm}^N f \, d\Omega, \quad (9)$$

with

$$Y_{lm}^N(\theta, \phi) = \left(\frac{2l+1}{4\pi} \right)^{1/2} d_{Nm}^l(\theta) e^{im\phi}, \quad (10)$$

where d_{Nm}^l is the real Wigner d -function. These are orthonormal, $\int \bar{Y}_{lm}^N Y_{l'm'}^N \, d\Omega = \delta_{ll'} \delta_{mm'}$. The sum starts at $l = |N|$ because d_{Nm}^l vanishes identically below it: there is no harmonic of that upper index at lower degree, and correspondingly no coefficient to store. This is the single fact most likely to look like a bug on first contact — see §9.3.

The normalisation is pinned by the code. Taking f constant, (9) gives $f_{00}^0 = 2\sqrt{\pi}f$ and $f = f_{00}^0/(2\sqrt{\pi})$, and `GaussLegendreGrid` implements exactly that on its one-point grid.

Conjugation. Using $d_{-N,-m}^l = (-1)^{m-N} d_{Nm}^l$, which is D&T (C.118), and the reality of d , equation (10) gives D&T (C.109),

$$\bar{Y}_{lm}^N = (-1)^{m+N} Y_{l,-m}^{-N}, \quad (11)$$

from which follow the two spectral reality relations of §5.3.

4 The generalized Legendre functions

$P_{lm}^N(\mu) = d_{Nm}^l(\theta)$, $\mu = \cos \theta$, is the regular solution of the generalized Legendre equation, normalised as D&T (C.112) so that the addition theorem below holds. It does *not* reduce to the associated Legendre function at $N = 0$; D&T (C.113) relates them.

4.1 Symmetries

D&T (C.118), in full:

$$\begin{aligned} P_{l,-m}^N(\mu) &= (-1)^{l+N} P_{lm}^N(-\mu), & P_{lm}^{-N}(\mu) &= (-1)^{l+m} P_{lm}^N(-\mu), \\ P_{l,-m}^{-N}(\mu) &= (-1)^{m+N} P_{lm}^N(\mu), & P_{lN}^m(\mu) &= (-1)^{m+N} P_{lm}^N(\mu), \\ P_{l,-N}^{-m}(\mu) &= P_{lm}^N(\mu). \end{aligned} \quad (12)$$

The third is the $(m, N) \mapsto (-m, -N)$ involution; the first is the $\theta \mapsto \pi - \theta$ reflection at fixed N , which maps the Gauss–Legendre node set to itself exactly, since those nodes are symmetric about $\pi/2$. Composed, the two generate a group of order four acting on (N, m, θ) , so the table of stored values could in principle be reduced fourfold. The matrix kernel uses the reflection to store only $m \geq 0$ (§13); the loop kernel’s table is not reduced, and §19 says why. The fourth relation is the transposition that the convention check discriminates against.

4.2 Boundary values

The values at the extreme orders have closed forms, D&T (C.123) and (C.125), and these are precisely the library’s `WignerMinOrder` and `WignerMaxOrder`:

$$P_{l,-l}^N = \left[\frac{(2l)!}{(l+N)!(l-N)!} \right]^{1/2} \left(\sin \frac{\theta}{2} \right)^{l+N} \left(\cos \frac{\theta}{2} \right)^{l-N}, \quad (13)$$

$$P_{ll}^N = (-1)^{l+N} P_{l,-l}^N|_{N \rightarrow -N}. \quad (14)$$

They are the seeds of D&T’s recursion in m , and the boundary terms of the library’s recursion in l (§10).

4.3 The addition theorem

D&T (C.127),

$$\sum_{m=-l}^l P_{lm}^N(\mu) P_{lm}^{N'}(\mu) = \delta_{NN'}. \quad (15)$$

This is the strongest available check on a computed table; `tests/CheckAdditionTheorem.hpp` is that check. Because the library stores X_{lm}^N rather than P_{lm}^N , the right-hand side there is $\delta_{NN'}(2l+1)/4\pi$.

5 Reality

Suppose $\mathbf{T}$ is a real tensor field. Writing $\mathbf{T} = \overline{\mathbf{T}}$ in (6) and using (2) on each factor,

$$\sum_{\alpha_1 \dots \alpha_p} T^{\alpha_1 \dots \alpha_p} \mathbf{e}_{\alpha_1} \otimes \dots = \sum_{\alpha_1 \dots \alpha_p} \overline{T^{\alpha_1 \dots \alpha_p}} (-1)^{\alpha_1 + \dots + \alpha_p} \mathbf{e}_{-\alpha_1} \otimes \dots, \quad (16)$$

and relabelling $\alpha_i \rightarrow -\alpha_i$ on the right gives the **reality condition**

$$T^{-\alpha_1 \dots -\alpha_p} = (-1)^N \overline{T^{\alpha_1 \dots \alpha_p}}, \quad N = \sum_i \alpha_i. \quad (17)$$

For a vector this reads $u^0 = \overline{u^0}$ and $u^- = -\overline{u^+}$: the radial component is a real field and the two transverse components are not independent.

5.1 Which components must be stored

Equation (17) pairs each multi-index with its *negation* $(\alpha_i) \mapsto (-\alpha_i)$, and one representative of each orbit of this involution suffices.

- The only fixed point of the negation map is the all-zero multi-index, which has $N = 0$ and, by (17), satisfies $T^{0\dots 0} = \overline{T^{0\dots 0}}$: it is a *real-valued* field.
- Every other multi-index lies in an orbit of size two; one member is stored as a complex field and the other is recovered by (17).

The storage cost in real numbers is therefore $1 + 2 \cdot \frac{1}{2}(3^p - 1) = 3^p$, which is precisely the number of real degrees of freedom of a real rank- p tensor in three dimensions. The reduction loses nothing and saves a factor of two against storing all 3^p components as complex fields:

rank p	components 3^p	stored fields	of which real	reals stored
0	1	1	1	1
1	3	2	1	3
2	9	5	1	9
4	81	41	1	81

Why “ $N \geq 0$ ” is not the rule. For a vector the orbits are $\{0\}$ and $\{+, -\}$, and selecting components with $N \geq 0$ picks exactly one from each. The coincidence fails at rank 2: $(+-)$ and $(-+)$ both have $N = 0$ and form a negation pair, so the criterion $N \geq 0$ would retain both. The selection rule must be stated on multi-indices, not on upper indices.

5.2 Interaction with permutation symmetry

If the tensor is symmetric (or antisymmetric) under some subgroup P of index permutations, the stored set is one representative per orbit of the group generated by P *together with* the negation map. A component whose P -orbit contains its own negation is pinned by (17) to a single real number per point — real, or purely imaginary if the permutation relating them carries a minus sign.

Such self-paired components always sit at $N = 0$: permutation preserves the slot sum while negation reverses it, so a component fixed by the combined operation satisfies $N = -N$. Since $(-1)^0 = +1$, a self-paired component of a *symmetric* tensor is always real; purely imaginary components arise only in the antisymmetric case.

Worked check: symmetric real rank 2. The six independent symmetric components are $(--), (-0), (-+), (00), (0+), (++)$. Under the combined group,

orbit	size	stored as
$\{(00)\}$	1	real field
$\{(-+), (+-)\}$	2	real field (self-paired, $N = 0$)
$\{(--), (++)\}$	2	complex field
$\{(-0), (0-), (+0), (0+)\}$	4	complex field

giving $1 + 1 + 2 + 2 = 6$ real numbers per point, the number of independent entries of a real symmetric 3×3 matrix. That the orbit construction reproduces this without special-casing is a check that the abstraction is the right one.

The general counts , computed by the library (§17.4) rather than tabulated by hand, are the independent real degrees of freedom in every case:

	complex components	pinned	reals per point
rank 1	1	1	3
rank 2	4	1	9
rank 2, symmetric	2	2	6
rank 2, antisymmetric	1	1	3
rank 3, symmetric	4	2	10
rank 4	40	1	81
rank 4, elastic	8	5	21

The last row is worth noting: nothing in the machinery knows about elasticity, and $c_{ijkl} = c_{jikl} = c_{ijlk} = c_{klij}$ comes out at the familiar 21.

5.3 Reality in the spectral domain

The same relation takes two distinct forms in the spectral domain, and it is worth keeping them apart.

Basis level. For *any* field f of upper index N , (11) gives the coefficients of $\bar{f}$, which has upper index $-N$:

$$(\bar{f})_{l,-m}^{-N} = (-1)^{m-N} \overline{f_{lm}^N}. \quad (18)$$

This is the relation exercised by `TestRealFieldSymmetry`.

Component level. For a negation-paired component of a *real tensor*, (17) supplies a further $(-1)^N$ and the two combine to

$$T_{l,-m}^{-N} = (-1)^m \overline{T_{lm}^N}, \quad (19)$$

which at $N = 0$ is the familiar condition $f_{l,-m} = (-1)^m \overline{f_{lm}}$ on a real scalar field, and is what the reduced $m \geq 0$ coefficient storage implements.

6 Raising and lowering

∂ and $\bar{\partial}$ raise and lower the upper index by one. On a basis function,

$$\partial Y_{lm}^N = -\sqrt{(l-N)(l+N+1)} Y_{lm}^{N+1}, \quad \bar{\partial} Y_{lm}^N = +\sqrt{(l+N)(l-N+1)} Y_{lm}^{N-1}, \quad (20)$$

so in the spectral domain each is a multiplication by an l -dependent factor, with no coupling between different (l, m) . They are the operations that connect different upper indices, and they are diagonal in (l, m) rather than local in (θ, ϕ) — which is why the library has two representations rather than one, and why an expression such as “the gradient of a product” is necessarily evaluated in both: the product is local in (θ, ϕ) , the gradient in (l, m) .

The surface gradient of a scalar has canonical components proportional to ∂f and $\bar{\partial} f$, at upper index $+1$ and -1 , with vanishing radial component; surface divergence and curl of a vector field are the corresponding contractions.

Two identities, and what fixes the rest. Composing the factors,

$$\bar{\partial}\partial|_{N=0} = -l(l+1), \quad [\bar{\partial}, \partial] = -2N. \quad (21)$$

The first says $\bar{\partial}\partial$ is the surface Laplacian on a scalar; it pins the relative sign of the two operators and both magnitudes. The second is structural. Both are tested, and both survive flipping the sign of the pair, so neither fixes the overall sign. That sign is not a question for the literature — P&B do not use $\bar{\partial}$ as such — but is fixed by requiring gradients of functions to come out as gradients in the physical frame, which ties it to the sign in (1): see §1.4. The two are one convention, stated in this document and observed by `tests/TestConventions.cpp`.

The degree ranges look after themselves. Raising a field at $N \geq 0$ produces one starting at $l = N + 1$, and the factor at $l = N$ is $\sqrt{0} = 0$: the coefficient with nowhere to go was going to vanish. Lowering produces a field starting at a lower degree, whose extra coefficient has no source and is zero, which is what having no content there means.

7 The contravariant derivative, and how it differs

Phinney & Burridge, and Dahlen & Tromp following them, work not with $\bar{\partial}$ but with a *contravariant derivative* ∂^σ , $\sigma \in \{-, 0, +\}$, which takes the expansion coefficients of a rank- q tensor to those of the rank- $(q+1)$ tensor $\nabla \mathbf{T}$. It is closely related to $\bar{\partial}$ and it is not the same operator, and the difference matters for anyone using this library on tensors.

7.1 The definition

With D&T (C.146),

$$\Omega_l^{\pm N} = \sqrt{\frac{1}{2}(l \pm N)(l \mp N + 1)}, \quad (22)$$

their (C.151)–(C.153) read

$$\partial^- T_{lm}^{\alpha_1 \dots \alpha_q} = r^{-1} \left[\Omega_l^{+N} T_{lm}^{\alpha_1 \alpha_2 \dots \alpha_q} - T_{lm}^{(\alpha_1-1) \alpha_2 \dots \alpha_q} - \dots - T_{lm}^{\alpha_1 \alpha_2 \dots (\alpha_q-1)} \right], \quad (23)$$

$$\partial^0 T_{lm}^{\alpha_1 \dots \alpha_q} = \frac{dT_{lm}^{\alpha_1 \dots \alpha_q}}{dr}, \quad (24)$$

$$\partial^+ T_{lm}^{\alpha_1 \dots \alpha_q} = r^{-1} \left[\Omega_l^{-N} T_{lm}^{\alpha_1 \alpha_2 \dots \alpha_q} - T_{lm}^{(\alpha_1+1) \alpha_2 \dots \alpha_q} - \dots - T_{lm}^{\alpha_1 \alpha_2 \dots (\alpha_q+1)} \right], \quad (25)$$

with the stipulation that any coefficient whose shifted index leaves $\{-1, 0, 1\}$ is zero. The components of the gradient are $(\nabla \mathbf{T})^{\sigma \alpha_1 \dots \alpha_q} = \partial^\sigma T^{\alpha_1 \dots \alpha_q}$: the operator *prepends a slot*, and the result carries upper index $\sigma + N$, which is (7) applied to the lengthened multi-index.

7.2 Two differences, one benign and one not

A factor of $\sqrt{2}$, which is only the basis normalisation. Comparing (22) with (20),

$$|\text{raising factor}| = \sqrt{(l-N)(l+N+1)} = \sqrt{2} \Omega_l^{-N}, \quad |\text{lowering factor}| = \sqrt{2} \Omega_l^{+N}. \quad (26)$$

The $\sqrt{2}$ is exactly the $1/\sqrt{2}$ in $\mathbf{e}_\pm = \mp(\hat{\boldsymbol{\theta}} \pm i\hat{\boldsymbol{\phi}})/\sqrt{2}$, which appears explicitly in D&T's surface gradient (C.144). So on the unit sphere, up to that factor and to the sign convention of §6, ∂^+ is $\bar{\partial}$ and ∂^- is $\bar{\partial}$ — *for a scalar*.

Connection terms, which are not benign. For $q \geq 1$ the bracket in (23) and (25) is not a multiplication. It subtracts the components with one slot index shifted, once per slot. Those terms are the connection: the canonical basis vectors themselves vary over the sphere,

$$[\pm\partial_\theta + i(\sin\theta)^{-1}\partial_\phi] \mathbf{e}_\alpha = \alpha \cot\theta \mathbf{e}_\alpha - \sqrt{2} \mathbf{e}_{\alpha\pm 1}, \quad (27)$$

so differentiating a tensor field differentiates its basis too. A scalar has no slots and therefore no connection terms, which is why the two operators coincide there and only there.

7.3 The consequence for this library

Applying $\tilde{\partial}$ to each component of a tensor does not give the gradient of the tensor. Use `SurfaceGradient`. Component-wise $\tilde{\partial}$ gives the $\tilde{\partial}$ of each component as a spin-weighted function, which for $q \geq 1$ is not a component of $\nabla \mathbf{T}$ and in general is not a component of anything.

This is a trap rather than a limitation: `Raise` and `Lower` are correct for what they claim — a single spin-weighted function — and are correct for the surface gradient of a scalar, which is the case most people reach for first. They are simply not the *ambient* tensor operator, and nothing in their types says so, because a tensor expansion's components are ordinary spin expansions.

7.4 The intrinsic derivative, where $\tilde{\partial}$ is exactly right

The paragraph above must not be read as forbidding component-wise $\tilde{\partial}$ altogether, because there is a case in which it is precisely the correct operator.

There are two derivatives on the sphere. ∂^σ is the *ambient* one: it differentiates the tensor and its basis, and the basis leaves the tangent plane. The *intrinsic* covariant derivative — the Levi-Civita connection of the induced metric, which is what surface differential geometry means by differentiation — is closed on *tangential* tensors, those whose every slot is ± 1 with no radial index.

On such a tensor the connection terms vanish identically. Every shift $\alpha_i + \sigma$ with $\alpha_i, \sigma \in \{\pm 1\}$ lands on $-2, 0$ or $+2$; the outer two leave the alphabet, and 0 names a component with a radial slot, which a tangential tensor does not have. What is left is pure Ω multiplication. Splitting $\nabla_1 T$ by whether the inherited slot is radial,

$$(\nabla_1 T)^{\sigma\alpha_1\ldots\alpha_q} = \Omega_l^{\mp N} T^{\alpha_1\ldots\alpha_q}, \quad \text{all } \alpha_i \text{ tangential}, \quad (28)$$

$$(\nabla_1 T)^{\sigma\alpha_1\ldots 0_j \ldots \alpha_q} = -T^{\alpha_1\ldots \sigma_j \ldots \alpha_q}, \quad \text{one slot radial}. \quad (29)$$

Both hold *exactly*, and are checked as such against `SurfaceGradient` applied to a rank-1 tensor with its radial component set to zero.

(29) is the extrinsic curvature: on a unit sphere the normal part of $\nabla_a v_b$ is exactly $-v$. (28) says

On a tangential tensor the intrinsic covariant derivative is $\tilde{\partial}$ applied component by component, up to $\sqrt{2}$, and it is closed.

So the two operators differ by an algebraic term, not a differential one, and $\tilde{\partial}$ is not a poor relation of ∂^σ : it is the intrinsic derivative, exact on exactly the objects — tangential, spin-weighted — that it was invented for. Which of the two a caller wants depends on whether the radial directions are part of the problem.

7.5 What is implemented

`SurfaceGradient` is ∇_1 : (23) and (25) without the r^{-1} , taking a rank- q tensor expansion to a rank- $(q+1)$ one.

No radial part. ∂^0 is $\partial/\partial r$, which an angular library cannot supply; but the *surface* gradient has no $\mathbf{e}_0$ component at all (D&T C.145), so the operator is complete without it and the radial derivative is the user’s to provide. What that means for the result is that its $\sigma = 0$ block is stored and zero: a rank- $(q+1)$ tensor whose $\mathbf{e}_0$ slot vanishes is an ordinary tensor, and keeping it as one leaves the result composable with contraction, further gradients and the transform, for a third of the storage.

The result has no symmetry and keeps the operand’s reality. ∇_1 of a symmetric tensor is not symmetric in the new slot against the old ones, and inferring a symmetry from an operator is a problem **Materialise** does not attempt. ∇_1 of a real tensor is real, so only the reduced set is computed.

It reads components that need not be stored. Every term is a coefficient of some component at fixed (l, m) , and the shifted components are in general not the stored representatives. They are read through **TensorExpansion::Coefficient**, which answers for any component of the tensor (§17.5).

What the tests pin. On a scalar the operator is ∂ up to $\sqrt{2}$, which checks the Ω factors and nothing else. The metric trace of the *second* gradient of a scalar is the surface Laplacian, $-l(l+1)$ — and that runs through the connection terms, since ∂^- of the $+1$ component subtracts the 0 component, so the identity fails outright if the operator is ∂ applied component-wise. The chain rule (30) is checked at rank one, in the spatial domain. And the gradient of a real tensor is checked against the same field widened to a complex one, which exercises reading derived components and writing pinned ones.

Divergence and curl are contractions of the gradient (D&T C.6.2). Contraction is pointwise, so a caller evaluates the gradient and contracts with the machinery of §17; there is no separate operator.

One further property worth knowing. ∂^σ preserves the chain rule as an operator on fields, D&T (C.154)–(C.155): $\nabla(\mathbf{T}\mathbf{P}) = (\nabla\mathbf{T})\mathbf{P} + \mathbf{T}(\nabla\mathbf{P})$ gives

$$\partial^\sigma(T^{\alpha\dots}P^{\beta\dots}) = (\partial^\sigma T^{\alpha\dots})P^{\beta\dots} + T^{\alpha\dots}(\partial^\sigma P^{\beta\dots}). \quad (30)$$

It does *not* hold coefficient by coefficient, because a product of fields is not a product of coefficients — which is the same fact that makes “the gradient of a product” an operation in two representations.

8 Quadrature

Colatitude. Gauss–Legendre with $n_\theta = l_{\max} + 1$ nodes, which integrates a polynomial of degree $2n_\theta - 1$ in $\cos\theta$ exactly. The nodes are the roots of P_{n_θ} , mapped by $\theta = \arccos(-x)$, and are symmetric about $\pi/2$.

Longitude. The trapezoid rule on n_ϕ equally spaced points, which is exact for $e^{i(m-m')\phi}$ only when $|m - m'| < n_\phi$. Analysing a band- $l_{\max}$ field needs $|m|, |m'| \leq l_{\max}$, hence

$$n_\phi \geq 2l_{\max} + 1. \quad (31)$$

At $n_\phi = 2l_{\max}$ the orders $m = +l_{\max}$ and $m = -l_{\max}$ are the same discrete mode and cannot be separated. The library takes the smallest *fast* FFT length at or above the bound — restricted

to $2^a 3^b 5^c 7^d 11^e 13^f$ with $e + f \leq 1$, the set FFTW has codelets for — because the naive $2l_{\max} + 2$ lands on $514 = 2 \times 257$ at $l_{\max} = 256$, and 257 is prime. The smooth choice there is 520, at about 1.5% more FFT work, and the FFT is not the expensive stage (§9).

Headroom. A product of two band- L fields has band $2L$, and $\int |f|^2$ for band- L f integrates a degree- $2L$ quantity. Both want a grid resolving more than the fields do. `Grid::ForBand(L, nMax, oversampling)` expresses that distinction — the band of the fields against the resolution of the grid — rather than leaving a caller to compute a degree and hope; the $3/2$ dealiasing rule is $\text{oversampling} = 1.5$ and 2 is exact for a single product.

Part II

Numerical methods

9 The transform

9.1 The algorithm

The forward transform of a field at upper index n is

$$f_{lm}^n = \sum_i w_i X_{lm}^n(\theta_i) F_m(\theta_i), \quad F_m(\theta) = \frac{2\pi}{n_\phi} \sum_j f(\theta, \phi_j) e^{-im\phi_j}, \quad (32)$$

so it is an FFT in longitude at each colatitude, followed by a weighted sum over colatitudes against the stored X . The inverse runs the same two stages in the other order. Both are separable exactly because (10) is a product of a function of θ and $e^{im\phi}$.

The Legendre stage has two implementations, carried permanently and selected per grid by `TransformKernel::Loop()`, which accumulates (32) directly, colatitude by colatitude; and `Matrix()`, which reorganises it as a set of matrix products and is described in §13.

Cost. The FFT stage is $O(n_\theta n_\phi \log n_\phi)$ and the Legendre stage is $O(n_\theta l_{\max}^2)$, and at production sizes the second dominates completely. Measured at $l_{\max} = 256$, $n = 2$, with the loop kernel on one thread: the FFT stage takes **0.43 ms** and the Legendre stage **18.6 ms** — a factor of 44. Any optimisation effort that does not address the Legendre stage is misdirected, and this is why the library’s performance work concentrates entirely there.

What binds the loop kernel is less obvious than it looks. Single-threaded it runs at about 19% of the read bandwidth one core can achieve, so it is not DRAM-bound; what binds it is load/store throughput. Per stored value the inner loop does one 8-byte read that reaches memory and then, from cache, a read of the FFT bin and a read-modify-write of the coefficient — four memory operations, only one of which is a miss. Rearranging the loop (hoisting the weight, splitting real and imaginary parts, raw pointers) moves it by less than a factor of two in either direction, because none of those changes the ratio of arithmetic to memory operations. The matrix kernel does (§13.2).

9.2 Real fields

A real-valued field exists only at $n = 0$, where $f_{l,-m} = (-1)^m \overline{f_{lm}}$ by (19). The transform then uses a real-to-complex FFT and stores only $m \geq 0$, halving both the FFT work and the coefficient count. At $n \neq 0$ the same reduction is *invalid*: it assumes the self-relation $f_{l,-m}^N = (-1)^{m-N} \overline{f_{lm}^N}$, which holds only when f is its own conjugate, and a spin-weighted field

is its own conjugate only at $N = 0$. The library therefore forbids real-valued fields at nonzero upper index in the type system (§2.1), and the transform throws if asked.

9.3 Transforming is a projection

A component at upper index N has no content below degree $|N|$, because no harmonic there exists. So the forward transform of an *arbitrary* field discards whatever that field had at low degree, and evaluating the result does not reproduce the input. It reproduces its projection onto the span of the harmonics that exist.

This is not an error and not a loss of accuracy: it is what the expansion means. Once a field is band-limited — for instance because it came from an inverse transform — the round trip is an identity to rounding. Example 06 shows both, and prints 5×10^{-1} for the first and 4×10^{-14} for the second.

10 Computing the d -functions

This is where nearly all the arithmetic is, and where the library differs most from the reference texts.

10.1 Recursion in l , not in m

D&T describe two schemes, both at fixed degree: their (C.121) recurs in m at fixed N , and (C.122) recurs in N at fixed m . For stability they iterate (C.121) upward from $m = -l$ and downward from $m = +l$ and meet at $m = N \cos \theta$, seeding each end with the closed forms (13)–(14).

The library recurs in *degree*, at fixed (N, m) . This is the scheme Woodhouse used, and the code is an evolution of his Fortran 77 implementation; D&T do not discuss it. The recursion is

$$d_{Nm}^l = f_1 d_{Nm}^{l-1} - f_2 d_{Nm}^{l-2}, \quad (33)$$

with

$$f_1 = \frac{(2l-1)[l(l-1)\cos\theta - Nm]}{(l-1)\sqrt{(l^2 - N^2)(l^2 - m^2)}}, \quad f_2 = \frac{l\sqrt{((l-1)^2 - N^2)((l-1)^2 - m^2)}}{(l-1)\sqrt{(l^2 - N^2)(l^2 - m^2)}}. \quad (34)$$

Why this one. The loop kernel consumes the values one degree at a time, in ascending order, for a fixed (N, θ) — see (32). A recursion in l produces them in exactly that order, so the same loop that fills the table can feed the transform directly (§11), and a truncated transform can stop early and pay for nothing it does not use. A recursion in m produces a whole degree at once and would have to be run to completion first.

What it costs. Of the coefficients in (34), only the $\cos \theta$ term depends on the colatitude; the four square roots are θ -independent and are read from a precomputed table of $\sqrt{k}$ and $1/\sqrt{k}$ — two vectors of $2l_{\max} + 1$ entries, which is the whole of the auxiliary storage the recursion needs. The inner loop runs over m at fixed (l, N, θ) with unit stride on three rows, which vectorises.

The removable singularity. At $l = |N| + 1$ the factor $1/(l-1)$ in (34) is $1/|N|$, which is a division by zero at $N = 0$; but $N/(l-1) = \text{sign}(N)$ there, and d^{l-2} does not exist, so that degree is taken by a separate one-term branch with the singular factor cancelled analytically. The same one-term form appears at the “critical degree” $l = m_{\max} + 1$, where the row stops growing.

10.2 The seed row and the boundary terms

The recursion needs starting values in two senses.

The seed row , at $l = |N|$, is a loop over m at a single degree. At $l = |N|$ the closed form reduces to

$$P_{l,-m}^N|_{l=|N|} = \sqrt{\binom{2l}{l+m}} \left(\sin \frac{\theta}{2}\right)^{l-m} \left(\cos \frac{\theta}{2}\right)^{l+m}, \quad (35)$$

and the binomial row is generated exactly by the standard step $\binom{2l}{k} = \binom{2l}{k-1}(2l-k+1)/k$. The row is at most $2|N|+1$ long — five entries for a rank-2 application — so the choice is not about cost; see §10.4.

The boundary terms are the values at $m = \pm l$, needed once per degree as the row grows. Evaluating (13) directly costs three $\log \Gamma$ and an exponential each time, which at $l_{\max} = 256$ is about 1.3×10^5 transcendental evaluations per upper index per pass. They obey a one-term recursion in l : writing $s = \sin(\theta/2)$, $c = \cos(\theta/2)$,

$$\frac{P_{l,-l}^N}{P_{l-1,-(l-1)}^N} = \sqrt{\frac{2l(2l-1)}{(l-N)(l+N)}} sc, \quad (36)$$

valid for $l > |N|$, seeded at $l = |N|$ where the square root is one and the value is $s^{2|N|}$ for $N \geq 0$ and $c^{2|N|}$ for $N < 0$. Both boundaries ride on the same recursion, because the ratio depends on N only through N^2 and because $P_{ll}^N = (-1)^{N+l} P_{l,-l}^{-N}$.

10.3 Why the recursion is the safer form

The closed form (13) has to be evaluated in logarithms: $\sqrt{(2l)!/\cdots}$ is of order 4^l and overflows long before the value it belongs to, which is bounded by one. The recursion (36) multiplies by a factor of order $2sc = \sin \theta \leq 1$ at each step, so nothing large is ever formed and no $\log \Gamma$ is needed. It is more robust than the closed form, not less.

Measured against the closed forms over $l_{\max} = 128$, $|N| \leq 3$ and six colatitudes, the worst relative discrepancy is 2×10^{-13} in double and 9×10^{-17} in long double — very nearly the same multiple of ε , about 1000, in both, and growing linearly in $l_{\max}$ rather than with the number of entries compared. That says the drift belongs to the *closed form*, which exponentiates a logarithm of size $O(l \log 4)$ and loses bits in proportion. The recursion's values are the better ones, and the test's tolerance is written as $20 l_{\max} \varepsilon$ for that reason.

What it cannot do is go on for ever. “Nothing large is ever formed” has a converse: something small is. The seed of the column at order m is about $(\sin \theta)^m$, and the column grows back to order one only once $l \sin \theta$ reaches m . A seed that has underflowed is harmless if its column never recovers within the table; it is fatal if the column does. Both at once need $m |\ln \sin \theta| > |\ln \min|$ with $m < l_{\max} \sin \theta$, that is $l_{\max} s |\ln s| > |\ln \min|$ for some $s = \sin \theta$, and $s |\ln s|$ is greatest at $s = 1/e$. So the recursion is sound if and only if

$$l_{\max} < e |\ln \min|, \quad (37)$$

which is 1926 in double precision and 237 in single. Measured as the defect of $\sum_m |d_{nm}^l|^2 = 1$ at $\theta = \arcsin(1/e)$, the table is right to 3×10^{-13} at $l_{\max} = 1830$, wrong in the ninth place at 1930 and in the third at 2000, the same for every $|n| \leq 4$; in single precision it is right at 240 and wrong at 250. Above the bound whole columns come back zero where the truth is of order one, and nothing says so.

The library therefore *refuses* a table above

$$\text{MaxSafeDegree} = \lfloor e(|\ln \min| - |\ln \varepsilon|) \rfloor, \quad (38)$$

which is 1827 in double and 30747 in long double — the $|\ln \varepsilon|$ keeping the seed above $\min/\varepsilon$ and so out of the denormal range, where it loses bits before it loses everything.

Single precision is recursed in double. Read naively, (37) with the $|\ln \varepsilon|$ margin would stop a single-precision grid at degree 194. But single precision earns its place in the transform, where it halves the table’s traffic and a field’s memory, and earns nothing in a recursion that runs once. So a `float` table is the `double` one, rounded: recursed in double into per-thread scratch and narrowed on store, by all three routes — the stored table, the matrix kernel’s transposed table, and the block a generating grid computes inside a transform — which is also what keeps the stored and generated paths agreeing bit for bit. The ceiling for `float` is then double’s. The recursion in double costs about one and a half times the recursion in single, paid once at construction.

10.4 The poles, and why no $\log \Gamma$ is evaluated on a threaded path

At $\theta = 0$ and $\theta = \pi$ the ratio (36) is $0 \times \infty$. No special case is needed: the seed is already the exact 0 or 1 that the closed form’s boundary branches return, and the factor is zero thereafter, so the recursion produces the right answer unaided.

The seed row (35) is generated from binomials rather than from the closed form for a different reason. `glibc’s lgamma` writes the global `signgam`, and the table is filled from every thread, so the closed form would be a data race there — benign, since nothing reads `signgam`, but a race. Generating the binomial row keeps every $\log \Gamma$ off the threaded paths, and the values are exact rather than merely accurate. It costs nothing measurable: the boundary is two values per degree out of $2l + 1$, so however expensive a $\log \Gamma$ is, it would be paid on under 1% of the table. (The closed forms remain in the library as `WignerMinOrder` and `WignerMaxOrder`, and as the oracle for the recursion.)

ThreadSanitizer cannot by itself confirm the absence of such races: GCC’s `libgomp` carries no TSAN annotations, so every OpenMP region is reported as racing with whatever ran before it. A Clang build against a `libomp` compiled with `LIBOMP_TSAN_SUPPORT` is needed for a real answer.

11 Storage and access

11.1 The table

For the loop kernel, values are held as a flat array indexed by (N, i_θ) then by (l, m) , contiguous in (l, m) for each (N, i_θ) — so a degree’s row is a contiguous run, which is what the loop’s inner loop wants. Offsets are closed form (`GSHIndices::OffsetForDegree`), not looked up. The matrix kernel stores the same values in a different order (§13.3).

The table is large. At $l_{\max} = 256$ with $n_{\max} = 2$ and all upper indices it is $5 \times 257 \times (257^2 - N^2)$ doubles, about 648 MB; at $l_{\max} = 512$ it is 5.4 GB. This is why the grid is a value-semantic handle over shared immutable state: copying one is a pointer copy plus a few scalars (the chunking policy and the FFTW planner flag, which are read per call and decide nothing about the table), and never a copy of the table. Two grids sharing an implementation report the same `Identity()`, which is the test the field layer makes when it combines fields.

11.2 The row supplier

Both loops of the loop kernel take the row for a degree from an accessor rather than walking a single iterator across the whole block. The only thing either loop needs is *a contiguous run of values in (l, m) order for one (N, i_θ)* , and asking for it one degree at a time costs a few integer operations against a loop of length $(2l + 1)k$.

That seam lets the values come from somewhere other than a table. A second implementation runs the recursion into per-thread scratch inside the transform, chosen at grid construction as `WignerValues::Generated()`. It builds no table at all: construction at $l_{\max} = 256$ goes from 0.248s and 648 MB to 0.015s and nothing.

It is slower, and by a ratio threads do not change. On an 8-core laptop the generated path is 1.4–3.2 $\times$ slower than the stored one in every configuration measured. Batching narrows the gap, since generation amortises over a chunk exactly as a table stream does, but *threads do not*: the ratio at eight threads is the ratio at one. So the stored path is not pinned at a memory roof that generation could escape by using more cores, at least on eight cores, and a machine with more memory bandwidth per core favours the stored path further. What the generated path offers is the memory saving, which is large, and possibly an advantage on a multi-socket machine where a table must be read across NUMA domains; only such a machine can measure that. It does not combine with the matrix kernel (§13.6).

12 Batching, chunking and threading

12.1 The batch

The transform’s primitive is “transform k fields sharing a grid, a degree and an upper index”, with the single field as $k = 1$. What is being amortised is the Wigner block for that upper index, which is why fields at different upper indices cannot batch together — for a rank-2 tensor that means batching over radii within each N , not across components.

A batch is described by (count, stride, dist) rather than by contiguity: element j of field k lives at $j \cdot \text{stride} + k \cdot \text{dist}$. This is FFTW’s advanced-interface form, and it costs nothing at the stage that actually reads the caller’s layout, so it covers both arrangements the field layer produces:

	stride	dist
components stored component by component	1	FieldSize
components stored point by point	$n_{\text{components}}$	1

The descriptor’s uniformity is a constraint on tensor storage. Its members must be evenly spaced, and in flat multi-index order the stored components sharing one upper index are scattered at no fixed spacing as soon as there is any symmetry. So a tensor’s buffer is ordered *by upper index*, which makes each group a contiguous run on both the field side and the coefficient side. Ordered any other way, the batching the primitive exists for would be unreachable from the tensor layer.

What batching is worth to the loop kernel. About 2.3 $\times$ at the optimum, not k . The gain saturates because the loop is bound by per-element load/store work rather than by table bandwidth, and batching removes $(k - 1)/k$ of the *table* reads while leaving that untouched. It also *reverses* beyond an optimum: the coefficient array is k times larger, and once it stops fitting in cache it is streamed from DRAM too.

12.2 Chunking

Because of that reversal, a batched call processes a large batch in chunks of an internally chosen size rather than handing the caller’s k straight to the inner loop. The chunk is

$$\text{chunk} \approx \frac{\text{cache}}{2 \times \text{copies} \times \text{bytes per coefficient block}}, \quad (39)$$

rounded to nearest and capped at 63, the factor of two being headroom for the streamed Wigner row and the FFT output. Rounding to nearest matters because the quotient lands just under an integer at the sizes that matter — truncation would turn 1.98 into a chunk of one, the collapse the rule exists to avoid. The cap is 63 rather than 64 because a chunk of 64 complex fields is an output stride of 1024 bytes, which triggers the power-of-two cache-set hazard of §13.3.

“Copies” is not the thread count in both directions. The forward transform gives every thread a private accumulator of $\text{chunk} \times \text{coefficientSize}$, so its copies are its threads; the inverse gathers one block, shared and read-only, so it has one copy however many threads read it. Using the thread count for both starves the inverse — at $l_{\max} = 256$, $k = 8$ on eight threads it would give a chunk of one where the whole batch fits, at about twice the cost.

On a multi-CCD or multi-socket machine a shared block is pulled into each cache domain that touches it, so the inverse’s single copy is really one per domain; the rule is optimistic there in a way it is not on a single shared cache. The chunk can be stated explicitly, or chosen by measurement (§16).

12.3 Threading

Threading is an explicit per-call policy and defaults to sequential. The library creates threads because it was asked to and not because it can, and the rule it keeps is that **exactly one level threads**: an operation asked to run in parallel from inside an existing parallel region runs sequentially instead, so a caller parallelising over slices, components or realisations cannot nest with the transform, and neither can the table build.

In the loop kernel the decomposition is over colatitudes, and the two directions differ. The inverse transform’s colatitudes write disjoint rows and share only read-only input, so they divide with no reduction at all. The forward transform’s colatitudes all contribute to every coefficient, so each thread accumulates into a private buffer and the partial sums are added at the end — partitioned rather than serialised: each thread sums every thread’s partials for its own block of coefficients. The matrix kernel decomposes over orders instead and needs no reduction in either direction (§13.4).

Measured , loop kernel, unbatched, at $n = 2$, complex, per call:

	1 thread	2	4	8	16
$l_{\max} = 128$, forward	1.75 ms	1.89×	2.98×	4.02×	3.13×
$l_{\max} = 128$, inverse	1.77 ms	1.83×	3.07×	4.15×	4.57×
$l_{\max} = 256$, forward	12.16 ms	1.63×	2.51×	2.68×	2.23×
$l_{\max} = 256$, inverse	12.54 ms	1.69×	2.64×	2.98×	2.97×

Sixteen threads is not better than eight and is often worse: the machine has eight cores and sixteen hardware threads, and for memory-bound work the second thread on a core adds contention rather than throughput. Callers should ask for cores.

At $l_{\max} = 128$ on eight threads the stage reaches 39–40 GB/s against a measured machine roof near 42, so it is genuinely bandwidth-bound there and nothing further can be won without reducing the traffic itself.

13 The matrix kernel

`TransformKernel::Matrix()` computes the Legendre stage as a set of matrix products. It is available when the library is built with a BLAS (§13.6), and `Tuning.hpp` chooses it over the loop kernel by measurement (§16).

13.1 The Legendre stage is a matrix product

The loop kernel accumulates the forward Legendre stage colatitude by colatitude:

$$f_{lm}^n += w_i X_{lm}^n(\theta_i) F_m(\theta_i) \quad \text{for each } i, l, m. \quad (40)$$

Reordered so that the sum over θ is innermost, and at fixed upper index n , it is one matrix product *per order*:

$$f_{lm}^n = \sum_i D_{li}^{(n,m)} b_i^{(m)}, \quad D_{li}^{(n,m)} = X_{lm}^n(\theta_i), \quad b_i^{(m)} = w_i F_m(\theta_i). \quad (41)$$

Over a batch of k fields the right-hand side becomes a matrix and this is a GEMM,

$$\underbrace{(n_L(m) \times n_\theta)}_{D^{(n,m)}} \times \underbrace{(n_\theta \times k)}_{\text{weighted FFT output}} \longrightarrow \underbrace{(n_L(m) \times k)}_{\text{coefficients}}, \quad (42)$$

with $n_L(m) = l_{\max} - \max(|n|, |m|) + 1$ degrees surviving at that order. The inverse transform is the same matrix applied the other way round, $(n_\theta \times n_L) \times (n_L \times k)$, so *one stored matrix serves both directions* through a transpose flag.

At $l_{\max} = 256$ there are $2l_{\max} + 1 = 513$ such products per upper index, of heights n_L from 257 down to 1, all with $n_\theta = 257$ columns.

13.2 Why it is faster

The loop kernel is not short of arithmetic intensity against DRAM; it is short of *arithmetic per memory operation* (§9). In (40), one stored value X is real and one FFT bin and one accumulator are complex, so each visit costs, in doubles, one load of X , two of the bin, and a load and a store of the accumulator — seven accesses — for two fused multiply-adds, or four flops. That is about 0.6 flops per double touched, and no rewriting of the loop moves it.

A GEMM microkernel holds a tile of the output in registers and streams the two operands past it. With a tile of m_r rows by n_r columns, each step of the K loop loads $m_r + n_r$ doubles and performs $m_r n_r$ fused multiply-adds:

$$\frac{\text{flops}}{\text{double touched}} = \frac{2m_r n_r}{m_r + n_r} \approx 5.3 \quad \text{at } m_r = 4, n_r = 8, \quad (43)$$

and the accumulator never leaves the registers, so its traffic vanishes entirely. That is an order of magnitude better than the loop, on precisely the metric that binds it.

The DRAM traffic is unchanged. The table is still read once per batch. So the matrix kernel can gain little where the stage is bandwidth-bound and gains most in the batched regime, where the loop kernel runs well below the memory roof and far below compute peak. Since the field layer always batches, that is the regime that matters; a benchmark run unbatched will show the matrix kernel doing little, and be right.

Why not an order of magnitude. The shape is unfavourable. A GEMM reaches a large fraction of peak when all three dimensions are large; here $M = n_L$ is large only for small $|m|$, $K = n_\theta$ is a few hundred, and $N = 2k$ is between 2 and 16. Skinny- N GEMMs are the case general BLAS kernels handle worst, and the 513 products per upper index are individually small enough that call overhead is not negligible. So (43) bounds what is available rather than predicting it.

13.3 Storage layout and the Fourier stage

The table is transposed. (42) needs $D^{(n,m)}$ contiguous in (l, θ) for fixed (n, m) , which is the transpose of the loop kernel’s layout. `WignerMatrices` stores it as $[n][m][l][\theta]$, so each matrix is a contiguous row-major block with leading dimension n_θ ; summing $n_L(m) n_\theta$ over m gives the same $66\,045 \times 257$ doubles per upper index at $l_{\max} = 256$ as the loop table.

The reflection halves it. D&T (C.118) gives $d_{nm}^l(\pi - \theta) = (-1)^{l+n} d_{n,-m}^l(\theta)$, so the matrix at $-m$ is the matrix at $+m$ with its columns reversed and a sign applied to row l . On a grid whose colatitudes are symmetric about $\pi/2$ — which `WignerMatrices::Reflected` checks rather than assumes — only $m \geq 0$ is stored, which halves the table. It halves the table and not the arithmetic: the same $2l_{\max} + 1$ products of the same shapes are still done. Pairing $\pm m$ into one product of double the width, which would convert the symmetry into arithmetic, measures worse sequentially, so the symmetry is used for traffic and memory only — which is what the DRAM argument above predicts it can buy.

The Fourier stage runs first and lands order-major. (42) wants, for one m , a contiguous (θ, κ) block, so all the FFTs run before any of the Legendre stage rather than interleaved per colatitude. The intermediate is $n_\phi \times n_\theta \times k$ complex, which is the size of the field itself. The transpose is nearly free: an FFTW `plan_many` with output stride equal to the number of transforms and output distance one writes the result directly in $[m][\theta][\kappa]$ order.

Nearly free, with one hazard. When the number of transforms times 16 bytes is a power of two, the writes for successive orders collide in the same cache sets, and the stage costs four to eight times what it costs at two transforms more or fewer. The Fourier stage is therefore blocked over a few colatitudes per FFTW call rather than run in one call over all of them; a small block is also faster in general, by 1.7–2 $\times$, and wants eighty times less workspace, since a strided write over a few hundred bytes stays within a couple of cache lines and one over tens of kilobytes does not.

The GEMM is real. D is real while the data are complex, so there is no standard complex-times-real BLAS call — but none is needed. A complex $(n_\theta \times k)$ block with κ fastest *is* a real $(n_\theta \times 2k)$ block, since `std::complex` stores its parts adjacently. So the whole stage is a real GEMM with $N = 2k$, and the doubling helps most exactly where k is small.

13.4 Threading

The products at different m write disjoint outputs. So the decomposition is over orders, and it needs **no accumulator and no reduction in either direction**: the forward transform’s thread-private accumulators, which the loop kernel needs and which compete for cache as the thread count grows, do not exist. The answer does not depend on which thread took which order.

The work per order is not constant: $n_L(m)$ falls linearly in $|m|$, so a static split over orders is about twice as unbalanced as it looks; and a split weighted by n_L would also be wrong, since the inverse’s inner dimension *is* n_L and a GEMM’s efficiency falls with its inner dimension, making time superlinear in n_L . The schedule is therefore dynamic, which needs no cost model.

A thread’s share of the table is a set of whole (n, m) blocks, which is a better arrangement for NUMA than a colatitude split.

13.5 Measured performance

Batched, at $l_{\max} = 256$ on eight threads, the matrix kernel is **5.8 to 6.0 $\times$ faster forward and 2.9 to 3.2 $\times$ inverse** than the loop kernel, and stores half the table.

Sequential and threaded. Sequentially the forward gain is 3.0 $\times$. The larger threaded gain is not the GEMM doing better but the loop kernel doing worse: at that size the loop forward scales only 1.5 $\times$ from one thread to eight, because eight private accumulators of 8.4 MB compete for a 16 MB cache. The matrix kernel has no accumulator to collapse, so the threaded comparison measures the GEMM and the absence of that problem together.

Unbatched. At $l_{\max} = 256$, $k = 1$ on eight threads the gain is 1.20 $\times$ forward and 1.10 $\times$ inverse, and this is the only regime in which the matrix kernel is worth little. The reason is that the loop kernel there reaches 65% of the memory roof and the matrix kernel 100%: with the traffic unchanged, the most it can win is the difference.

13.6 What it composes with

It does not combine with on-the-fly generation. The recursion runs up degrees at fixed (n, θ) , producing values in exactly the order the loop table stores them and exactly the wrong order for $[n][m][l][\theta]$. This is not an inconvenience to be paid for with a transpose. The recursion’s output for one (n, θ) spans *every order at once*, so isolating the single (n, m) block a per-order product wants means either keeping all of it — which is the table, and not having one is the entire purpose of generation — or re-running the recursion once per order, $2l_{\max} + 1$ times over. There is no third way, so `Matrix()` with `WignerValues::Generated()` is refused at grid construction. Since the generated path is slower anyway (§11), nothing is lost.

The loop kernel is its oracle. A GEMM sums in whatever order its kernel chooses, so the matrix kernel cannot be bit-compared against the loop kernel, and its batched-against-unbatched tests are tolerance comparisons. In exchange, keeping both kernels permanently makes the loop kernel an oracle for the matrix kernel on identical inputs, which checks the layout, the indexing, the FFT ordering and the accumulation together. The two are not independent in the d values, since both read the same recursion — but that half is pinned separately, against the $l = 1$ table and against `std::sph_legendre`.

Which BLAS. The library has no other dependency that supplies a GEMM, so the BLAS is an optional dependency, found and not fetched, under the CMake option `GSHTTRANS_WITH_BLAS`, in the pattern of the optional `Interpolation` dependency. A build without it does not offer the matrix kernel, exactly as a build without `Interpolation` does not offer `SplineDerivative`. It is the library’s one non-header-only dependency, which is why it is an option and not a requirement.

14 Interpolation

`Interpolate(field, scheme)` returns a callable of (θ, ϕ) modelling `ScalarFunctionS2`, which is what a field constructor takes, so remeshing is one line. Three schemes: `Scheme::Spectral()` sums the expansion and is exact for a band-limited field at $O(l_{\max}^2)$ a point; `Scheme::Bilinear()`

and `Scheme::Bicubic()` are `Interpolation`'s rectilinear schemes over the samples, $O(1)$ a point and second and fourth order respectively.

Two facts about the sphere make this more than a forwarding call. The longitudes run $0, \dots, 2\pi - \Delta\phi$, so the last cell has nothing to interpolate against; and neither pole is a grid point, so a local scheme extrapolates there — exactly where $d_{Nm}^l \sim \sin^{|m|} \theta$ is most delicate. Both are fixed by handing the scheme a *padded* grid: a wrap column, which is exact since $\phi = 2\pi$ is $\phi = 0$, and two polar rows.

The polar rows are computed rather than guessed. At a pole all but one order vanishes:

$$f(0, \phi) = e^{+iN\phi} \sum_l f_{l,+N}^N \sqrt{\frac{2l+1}{4\pi}}, \quad f(\pi, \phi) = e^{-iN\phi} \sum_l (-1)^{l-N} f_{l,-N}^N \sqrt{\frac{2l+1}{4\pi}}. \quad (44)$$

The ϕ dependence is the point: a spin-weighted field at a coordinate pole is not single-valued, because $\mathbf{e}_\pm$ depends on the azimuth of approach, so a polar row is a *row* and a constant one would be wrong at every $N \neq 0$. Computing it needs the expansion, which is why building a local interpolant costs a forward transform and why its cheapness is per evaluation rather than per interpolant.

Measured, the padding works: the polar cells and the last longitude cell come out two to five times *better* than the interior at every resolution, so the regions the padding was built for are not the weak ones. A local scheme wants an oversampled grid — bicubic carries 13% error at the band limit, 7×10^{-4} at four times it and 4×10^{-5} at eight — and `ForBand` is how that room is asked for. One whole-grid remesh costs about what fifty spectral point evaluations cost, so past a few dozen scattered points the composite of the two beats either alone.

15 The 3-j symbols

`3j.hpp` tabulates the whole (m_1, m_3) plane at fixed degrees, with $m_2 = -(m_1 + m_3)$ fixed by the selection rule. The table is the primitive: a single symbol costs one.

The values come from the Schulten–Gordon recursion. The order recurrence

$$A(m_2) g(m_2) + B(m_2) g(m_2 - 1) + A(m_2 - 1) g(m_2 - 2) = 0 \quad (45)$$

has a growing and a decaying solution, with a classically allowed region between two forbidden ones. Recursing *inward* from a forbidden end follows the growing solution, so rounding error decays; outward follows the decaying one, so it grows exponentially. Each row is therefore built inward from both ends, matched where the halves overlap, and normalised from the unitary property $(2l_1 + 1) \sum_{m_2} g(m_2)^2 = 1$. No closed-form seed is needed and no factorial is ever formed.

Why not a simpler scheme. A one-directional recursion fails exponentially near *stretched* triangles — where one degree approaches the sum of the other two, which is the top of every coupling sum rather than an exotic corner — and Racah's closed form covers exactly that region and no more. Together they still leave a band uncovered; Schulten–Gordon covers all of it, at 1.1 to 1.3 times the cost of the one-directional recursion.

Three details make it practical.

Where the forward run stops. It stops where the *values* first fall, and the backward run comes up to meet it: recursing towards larger values is what is stable, so the values are the thing to watch, and their first fall is the edge of the allowed region where there is one and the top of the hump in a near-stretched row, which has none. SLATEC's `DRC3JM` watches the recurrence coefficient $|c_1|$ instead and stops at its first rise. That is a proxy — good where there is an allowed region, since $|c_1|$ is least near the middle of one, and wrong in a near-stretched row,

where its minimum is nowhere near the hump and one half is left to run downhill. Downhill costs digits, 10^{-10} where 10^{-14} is to be had; and where the halves end more than 10^{154} apart the join overflows and the table cannot be built at all, which with the proxy is the case for 44 of the 77 tables $(l, 2l, l)$ up to $l = 1000$.

The join is formed through `hypot`, so that the ratio of the halves is never squared.

The phase is *tracked* through the match rather than read back from $g(m_{2\max})$: a near-stretched row at high degree spans a dynamic range of 10^{201} , so the rescaling flushes that element to zero and the sign with it.

What checks it. Not the completeness relation, which the algorithm normalises by and which therefore holds by construction; and not the recurrence residual alone, which is homogeneous and so blind to an overall scale or sign. The check that sees everything is invariance under a cyclic permutation of the columns, which runs the recursion along entirely different lines: it agrees to 10^{-16} at $(200, 200, 200)$ and 10^{-15} at $(1000, 1000, 1999)$. Racah is kept in the test tree as an independent formula, restricted to entries where its alternating sum is short and it is exact — which *is* the stretched region, reached by whichever cyclic permutation puts the stretched degree third; against it the tables agree to the 10^{-12} that Racah’s own logarithms allow.

(45) is also checked at run time, so a table that has gone wrong is refused rather than returned. The tolerance of that check is measured and not chosen: the worst residual a good row leaves is two or three in units of $(n + 1)\varepsilon$ times the row’s scale, and the bound is fifty. A much looser bound — a thousand — would pass a single-precision table wrong by a third of its own scale.

Rows are computed in at least double precision whatever they are stored in. A three-term recurrence accumulates about $n^2\varepsilon$ along a row of n values however good the algorithm, which in single precision leaves two digits at $l = 450$; the row is a scratch buffer, so this costs the difference in its arithmetic and no memory.

16 Choosing a policy by measurement

Several of the library’s choices are properties of the machine rather than of the mathematics, and cannot be settled by reasoning: the Legendre kernel, and the chunk. `Tuning.hpp` times the alternatives on the caller’s own problem and returns *values* — never a configured grid, so nothing is substituted silently.

The chunking policy and the planner flag live on the grid handle rather than on the shared implementation, since neither decides the table; `With()` changes either for a pointer copy, and the result shares `Identity()` with its parent, so fields are interchangeable between them. A sweep over candidate chunks therefore never rebuilds the table.

Measured on an 8-core laptop: the kernel choice is worth 1.9 to $5.8\times$ and picks the matrix kernel in every configuration tried, while the chunk is worth at most $1.28\times$ and nothing at all in thirteen of eighteen. Both are cheap enough to run at start-up — about a second — so the results are not persisted: the measurement is cheaper than remembering it, at least up to around $l_{\max} = 512$, where building the tables stops being cheap. A candidate must beat the incumbent by ten per cent to displace it, that being the measured noise floor (§18).

Part III

The field algebra

17 What the type system enforces

The field algebra is built so that a statement which is not mathematically meaningful does not compile. In summary, the facts of Part I that it enforces are:

1. A component is labelled by a multi-index; its upper index is the signed sum (7) and is a compile-time quantity.
2. Addition requires equal upper indices; multiplication adds them; division requires a denominator of upper index zero.
3. Conjugation maps $N \mapsto -N$.
4. Real part, imaginary part, nonlinear functions and integration over the sphere require $N = 0$.
5. A real-valued component field can occur only at $N = 0$.
6. For a real tensor, the stored components are one per orbit of index negation, combined with any permutation symmetry; the all-zero component is real-valued, self-paired components are pinned, and the remainder are complex.

What follows is how, with the reason in each case.

17.1 The index algebra

Every node — an owning field, a view, or an expression — carries its upper index as a compile-time constant, and the operators enforce the table of §2.1:

expression	upper index	requires
$f + g, f - g$	N	equal upper indices
fg	$N_f + N_g$	—
f/g	N_f	$N_g = 0$
$\bar{f}$	$-N_f$	—
$ f , f ^2$	0	—
$\text{Re } f, \text{Im } f, F(f)$	0	$N_f = 0$
$\int f \, d\Omega$	—	$N_f = 0$

The constraints are **requires**-clauses on the free operators, so an unlawful combination is an overload-resolution failure at the call site rather than an error inside a node — which also means the negative cases can be tested.

17.2 Real-valuedness

A node may be real-valued *only* at $N = 0$. Real-valuedness is not preserved by the frame rotation, so it is not a property any component of any tensor can have elsewhere; a library able to represent one could represent something that does not exist. The constraint is closed under every operation in the algebra — Re , Im , $|f|$, $|f|^2$ and $F(f)$ all land at $N = 0$ precisely because that is the only place their results could be real — so it is checked once, on the concept, and never re-derived.

17.3 Laziness, and the aliasing theorem

Operators build expression nodes; nothing is evaluated until a value is read. Every node in the layer is **pointwise and index-preserving**: the value at (i_θ, i_ϕ) reads its operands only at (i_θ, i_ϕ) . Hence

$u = \text{expr}$ is safe when u occurs in expr , and needs no temporary.

That is a theorem about the node set rather than a property of any one operator, which is why the invariant is stated in the stronger “index-preserving” form: a re-indexing view — a longitude shift, a transpose of the grid — would be pointwise in the loose sense and would break it. Operations that are genuinely not pointwise (gradients, raising and lowering, radial derivatives) live in the spectral layer for this reason.

Operand storage follows the value category: an owning field passed as an lvalue is held by reference; everything else, views included, by value. The residual hazard — an lvalue field destroyed while an expression referring to it is alive — is the same one Eigen has and is not removable in C++.

17.4 Which components a tensor stores

Both reductions of §5 are computed by one `constexpr` walk of the group generated by the permutation symmetry together with, for a real tensor, the negation. Reaching a component twice by different routes is the interesting case rather than a failure: equating the two routes produces a constraint.

the two routes differ by	the orbit is
a sign, same conjugation	identically zero
conjugation, signs equal	real
conjugation, signs opposite	purely imaginary

The first arises without reality at all — it is the vanishing diagonal of an antisymmetric tensor. The others are the self-paired components, which always sit at $N = 0$, and are therefore exactly where a real-valued field is admissible.

17.5 Derived components in the two domains

On the spatial side a derived component is pointwise — $T^{-\alpha} = (-1)^N \overline{T^\alpha}$ — so a view plus an expression node gives it. The component accessor therefore cannot return one type: a stored component is a view, a symmetric-permutation relative is the same view, an antisymmetric relative is that view negated, a reality relative is its conjugate at the reversed upper index, and a vanishing orbit has nothing to return and is refused. Traversal over all 3^p components is therefore a compile-time loop, not a runtime one.

On the spectral side the corresponding relation is (19), $T_{l,-m}^{-N} = (-1)^m \overline{T_{lm}^N}$, which *reverses the order index*. That is not a view over anything, so on this side the accessor is a computation returning a value at one (l, m) : `TensorExpansion::Coefficient`. It answers for any component of the tensor, resolving the stored case, a permutation relative (with its sign), a reality relative by (19), a real block’s reduced $m \geq 0$ storage, a component pinned as imaginary, and a vanishing orbit, which returns zero — as do degrees below the component’s own $|N|$, where it has no content. A slot letter outside the tensor’s alphabet (a radial index on a tangential tensor) is refused rather than answered with zero, since it names a component of a different bundle.

18 Validation

What each oracle actually pins, since a test suite that all passes is uninformative about which facts are load-bearing.

oracle	what it pins
<code>CheckWignerConvention</code>	That the stored value is X_{lm}^N with the upper index first, against all nine $l = 1$ values of D&T (C.115). Discriminating: the four entries with $N - m$ odd distinguish d_{Nm}^l from d_{mN}^l .
<code>CheckLegendre</code>	The $N = 0$ row against <code>std::sph_legendre</code> , which fixes the orthonormal scaling and the Condon–Shortley phase.
<code>CheckAdditionTheorem</code>	(15) over all $ N , N' \leq 40$ at $l_{\max} = 40$, to 1000ε . The strongest single check on the recursion: boundary values feed the two-term recursion at every higher degree, so an error anywhere propagates into it.
<code>CheckWignerBoundary</code>	The boundary recursion and the binomial seed row against the closed forms, to $20l_{\max}\varepsilon$ — a tolerance argued from where the drift actually is (§10).
<code>CheckCoeff2Coeff</code>	Coefficient-to-coefficient round trips.
<code>TestConventions</code>	The sign convention of §1.4: a gradient read back into the physical frame is the gradient, and the trace of the gradient of a tangential vector is its divergence.
<code>TestRealFieldSymmetry</code>	(18) at $n = 2$ through complex transforms, and the rejection of real transforms at nonzero upper index.
batched transform tests	For the loop kernel, a batch of k against k separate calls, at <i>exact</i> equality: batching widens the inner loop without reordering any sum, so any difference at all would mean it had been restructured rather than widened. For the matrix kernel, a tolerance comparison against the loop kernel (§13.6).
generated-path tests	The generating supplier against the stored table, again at exact equality: same recursion, same order, same rounding, only the storage differs.
reality tests	(17) at every component of every tensor, and that the stored set costs 3^p reals per point.
$\tilde{\mathfrak{d}}$ tests	(21), both of them.

On measuring this library. The development machine’s clock scales under load, and the noise floor between back-to-back runs of identical code is about 10%. Comparisons must interleave the two versions in one process; figures from different sessions are worth less still. Within-run ratios — the stage split, the batching curve — are unaffected, because their terms are measured moments apart. The performance figures in this document were measured on an 8-core laptop and characterise the methods rather than any particular deployment.

19 Scope and limitations

What the library deliberately does not do, and why.

The fourfold symmetry reduction of the loop kernel’s table (§4) is not implemented. In the regime that matters it would buy little: batched, with a chunk that fits, the loop kernel runs at about a fifth of the memory roof and under a tenth of compute peak, so halving the traffic addresses neither binding constraint. Unbatched it would be worth roughly $2\times$, and batching already takes that. The matrix kernel does use the reflection, but for storage: it halves the table, not the arithmetic.

Reduced-precision storage of a double-precision table is in the same position, for the same reason; an experiment would also need a tighter round-trip oracle than the tests have before its result meant anything.

Polar truncation — skipping the (m, θ) pairs where d_{Nm}^l is negligible, since it falls off like $\sin^{|m|} \theta$ — makes the problem smaller rather than making the machine work harder, but measures at only about $1.15\times$ at $l_{\max} = 256$ for the rectangular trim a GEMM can take, and $1.40\times$ at $l_{\max} = 1024$ for the triangular ceiling. The decay is real but slower to a useful tolerance than it looks: at 10^{-15} , $\sin^{256} \theta$ is still above threshold over a third of the range. It is not implemented.

Degrees above MaxSafeDegree (§10) are refused rather than supported. The known cure, an integer exponent carried beside each column (Fukushima, *J. Geod.* 86, 2012), is not built: nothing the library is used for comes near the bound of 1827 in double.

Persisted tuning results — a keyed, fingerprinted store of what §16 measured, in the manner of FFTW’s wisdom — are not kept. Both tuners run in about a second, so there is nothing expensive enough to be worth remembering; FFTW’s wisdom is worth persisting because planning can take minutes.

Deeper forms of on-the-fly generation , fusing the recursion with the consumer or blocking it over colatitudes, would roughly halve its arithmetic. They are not built because the generated path is slower than the stored path on every configuration measured (§11), by up to $3.2\times$, so halving the arithmetic would not in general close the gap. What the path is for is memory, and that it delivers.

The matrix kernel with generated values is refused, because the two are not combinable (§13.6).

Component-wise $\tilde{\partial}$ on a tensor is not prevented by the types, because a tensor expansion’s components are ordinary spin expansions; §7 explains when it is and is not the right operator.